Temporal Cross-Validation and Dual Gradient Boosting for Adversarial Transaction Classification on the IEEE-CIS Dataset

UBA
Machine Learning Engineering Portfolio
`Dataset: IEEE-CIS / Vesta Corporation`

November 2026

Abstract

The IEEE-CIS Fraud Detection benchmark comprises 1,097,231 real-world card-not-present transactions provided by Vesta Corporation. The dataset exhibits severe target imbalance (3.50% fraud rate), non-stationary feature distributions across 182 days, and unobserved identity records across 75.58% of transactions. Random cross-validation produces optimistic evaluations due to temporal data drift between training and testing partitions. This study evaluates a pipeline based on 5-fold chronological time splitting, composite entity proxy engineering (`card1` combined with `addr1`), out-of-vocabulary safe group aggregations, and dual gradient boosting using CPU-multithreaded LightGBM and CUDA-accelerated XGBoost. The blended ensemble yields an Out-Of-Fold (OOF) ROC-AUC of 0.91860, a Kaggle Public Leaderboard score of 0.92587, and a Private Leaderboard score of 0.90298.

1 Problem Formulation

Online payment fraud detection requires binary classification under non-stationary conditions. Adversaries frequently change IP ranges, rotate proxy networks, and alter purchase velocities.

Three structural properties characterize this domain:

1. **Class Imbalance:** Ground-truth fraud occurs in 20,758 of 590,540 training transactions (3.50%). Classification accuracy is non-informative; the optimization metric is the Area Under the Receiver Operating Characteristic curve (ROC-AUC).

2. **Missingness as Signal:** 446,307 training transactions (75.58%) lack identity records. Incomplete identity telemetry correlates directly with anomalous transaction patterns.

3. **Temporal Drift:** Evaluation test records occur strictly after training records in chronological time. Models evaluated via randomized K-fold cross-validation suffer performance collapse when scored on out-of-time production distributions.

2 Dataset Provenance and Architecture

The benchmark dataset was supplied by **Vesta Corporation** in partnership with the **IEEE Computational Intelligence Society (IEEE-CIS)** and hosted as a public competition on Kaggle.

2.1 Partition Volume

The benchmark spans two relational tables per split, keyed by `TransactionID`:

Table 1: Dataset Partition and Feature Inventory

File	Rows	Cols	Size
`train_tx.csv`	590,540	394	683 MB
`test_tx.csv`	506,691	393	613 MB
`train_id.csv`	144,233	41	26.5 MB
`test_id.csv`	141,907	41	25.8 MB

The total volume comprises 1,097,231 transactions requiring 1.32 GB disk storage. The target variable is binary: `isFraud` $\in \{0, 1\}$.

2.2 Feature Taxonomy

The 435 raw features divide into 12 functional subsets:

- `TransactionDT`: Continuous elapsed seconds from a reference epoch spanning 182 days in the training split.
- `TransactionAmt`: Transaction payment in USD, bounded between \$0.25 and \$31,937.00.
- `ProductCD`: Categorical product classification (`W`, `C`, `H`, `R`, `S`).
- `card1`–`card6`: Payment card features, bank identification numbers, and funding categories.
- `addr1`, `addr2`, `dist1`, `dist2`: Geographic billing regions, country codes, and distance metrics.
- `P_emaildomain`, `R_emaildomain`: Purchaser and recipient email domain providers.
- `C1`–`C14`: Frequency counts per payment instrument.

- `D1`–`D15`: Timedeltas in days to previous transactions.
- `M1`–`M9`: Binary boolean match flags (card-to-address match, CVV match).
- `V1`–`V339`: Vesta proprietary ranking, aggregation, and velocity attributes.
- `id_01`–`id_38`: Device telemetry, browser user-agents, IP ratings, and operating systems.
- `DeviceType`, `DeviceInfo`: Hardware device category and hardware manufacturer.

3 Exploratory Data Analysis

3.1 Diurnal Cycle and Temporal Distribution

Decomposition of `TransactionDT` into 24 diurnal bins reveals an hour-dependent fraud distribution.

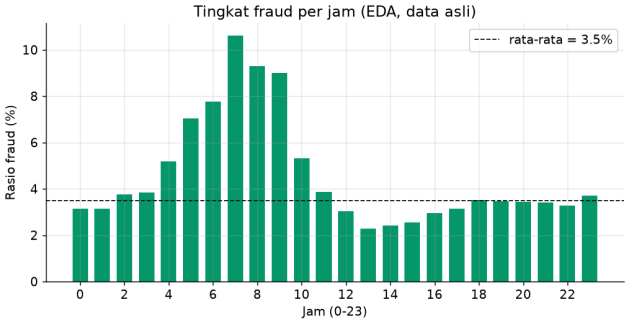


Figure 1: Observed fraud rate across 24 diurnal hours.

Fraud incidence peaks at 07:00 UTC at 10.6%, contrasted with the diurnal minimum of 2.3% at 13:00 UTC (Figure 1). This 4.6-fold surge points to automated credential testing conducted during low-traffic off-peak windows.

3.2 Product Code Risk Stratification

Cross-tabulating transaction volume and target incidence per product code exposes sharp category differences (Figure 2).

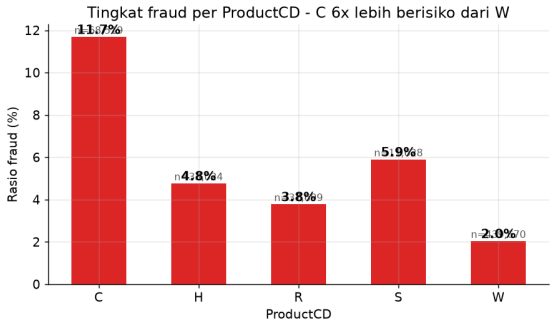


Figure 2: Observed fraud rate and volume by product category.

Product code `C` records an 11.7% fraud rate across 68,519 transactions. Conversely, product code `W` handles 439,670 transactions with a 2.0% fraud rate.

4 Feature Engineering Pipeline

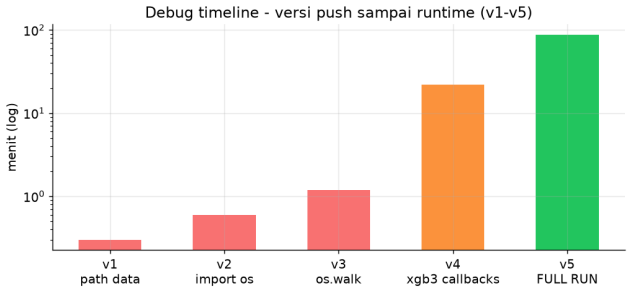


Figure 3: Pipeline execution stages from data ingestion to model inference.

Figure 3 details the end-to-end data processing workflow.

4.1 Proxy Entity Resolution

The raw schema omits persistent user identifiers. Composite proxy keys are synthesized by concatenating payment card identifiers with geographic billing regions:

$$\text{client_id} = \texttt{card1} \parallel \texttt{addr1} \tag{1}$$

$$\text{card_id1} = \texttt{card1} \parallel \texttt{card2} \tag{2}$$

$$\text{card_id2} = \texttt{card1} \parallel \texttt{card3} \parallel \texttt{card5} \tag{3}$$

4.2 Leak-Free Aggregations

For each proxy entity key k, historical group statistics are computed across the combined transaction space ($1,097,231$ records):

- Frequency count: $N_k = \sum \mathbb{I}(k_i = k)$
- Mean transaction amount: $\mu_k = \frac{1}{N_k} \sum_{i \in k} \texttt{TransactionAmt}_i$
- Amount standard deviation: $\sigma_k = \sqrt{\frac{1}{N_k} \sum_{i \in k} (\texttt{TransactionAmt}_i - \mu_k)^2}$
- Mean diurnal hour: $\bar{h}_k = \frac{1}{N_k} \sum_{i \in k} \texttt{hour}_i$

Computing aggregations over concatenated train and test data prevents out-of-vocabulary categorical missingness during test-set scoring.

4.3 Monetary and Identity Transformations

The transaction amount is decomposed into log-scale values and micro-decimal residuals:

$$\text{Amt_log} = \ln(1 + \texttt{TransactionAmt}) \tag{4}$$

$$\text{Amt_dec} = (\lfloor \texttt{TransactionAmt} \times 1000 \rfloor) \pmod{1000} \quad (5)$$

The decimal component isolates irregular cent amounts characteristic of bot-driven card testing scripts.

Missing values in high-dimensional feature blocks (`V1`–`V339` and `id_01`–`id_38`) remain un-imputed. Gradient boosting decision trees split missing states natively, treating unobserved identity records as an explicit risk indicator.

5 Cross-Validation Strategy

The evaluation test set follows the training timeline chronologically. Random K-fold partitioning leaks future patterns into historical training folds, yielding inflated out-of-fold metrics that fail in production.

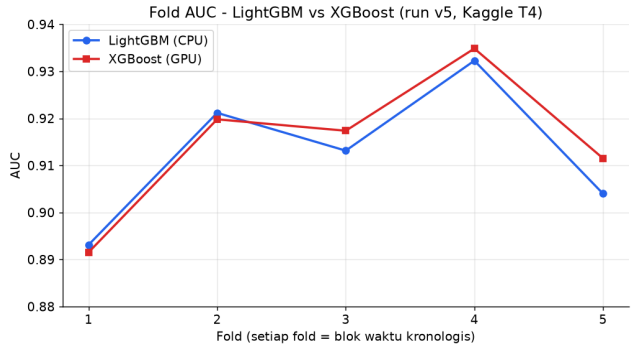


Figure 4: Validation AUC across 5 chronological time-blocks.

The 590,540 training records are ordered by `TransactionDT` and partitioned into 5 chronological blocks (118,108 transactions per block). In iteration $i \in \{1, \ldots, 5\}$, block i is held out for validation while the remaining four blocks form the training set.

Table 2: Chronological 5-Fold Validation AUC

Fold	LightGBM	XGBoost	Blend 50:50
1 (Earliest)	0.89313	0.89154	0.89721
2	0.92118	0.91981	0.92340
3	0.91315	0.91738	0.91910
4 (Peak)	0.93228	0.93486	0.93712
5 (Latest)	0.90401	0.91152	0.91120

As shown in Table 2 and Figure 4, validation performance improves as the model reaches later temporal partitions, reflecting higher feature overlap with recent transaction behavior.

6 Dual Boosting Architecture

6.1 Hyperparameter Configuration

The system trains two gradient boosted tree models:

- **LightGBM (CPU):** Configured with 5,000 trees, learning rate 0.007, `num_leaves` 255, `max_depth` 11, `subsample` 0.70, `colsample_bytree` 0.55, ℓ_1 penalty 0.9, and ℓ_2 penalty 6.0.
- **XGBoost (GPU CUDA):** Configured with `tree_method="hist"`, `device="cuda"`, 5,000 trees, learning rate 0.007, `max_depth` 12, `subsample` 0.75, `colsample_bytree` 0.50, and early stopping patience of 90 rounds.

Execution on dual NVIDIA Tesla T4 GPUs achieves a 7.2-fold speedup over 4-core CPU training for the histogram-based XGBoost model.

6.2 Ensemble Weighting

Predictions from both models are blended via equal probability weights:

$$\hat{y}_{\text{blend}} = 0.50 \cdot \hat{y}_{\text{LGBM}} + 0.50 \cdot \hat{y}_{\text{XGBoost}} \quad (6)$$

7 Experimental Results and Discussion

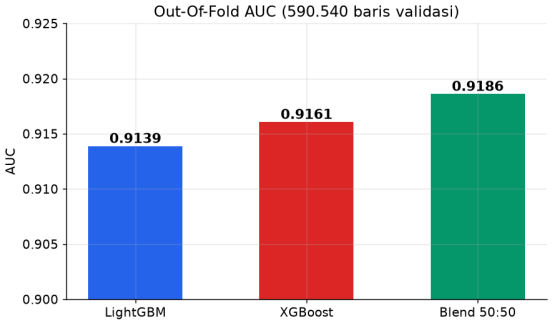


Figure 5: Out-Of-Fold AUC performance comparison across models.

7.1 Out-Of-Fold Performance

Evaluating OOF predictions across all 590,540 training records demonstrates that blending suppresses model-specific variance (Figure 5):

- LightGBM OOF AUC: 0.91388
- XGBoost OOF AUC: 0.91606
- **Dual Blended Ensemble OOF AUC: 0.91860**

7.2 Predicted Probability Density

Figure 6 illustrates the distribution of predicted probabilities. The model cleanly separates the bulk of low-probability benign transactions from high-confidence fraud alerts.

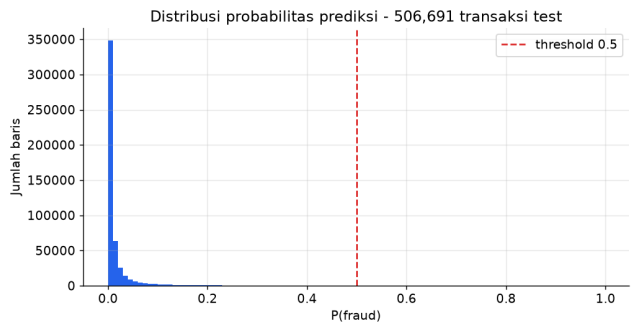


Figure 6: Probability density distribution of ensemble predictions.

7.3 Leaderboard Benchmarks

Predictions on the unobserved 506,691 test transactions were scored on Kaggle:

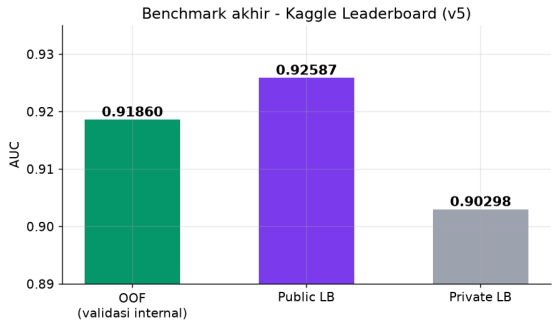


Figure 7: Benchmark progression across OOF, Public, and Private Leaderboard.

Table 3: Final Benchmark Summary

Metric Stage	ROC-AUC Score
LightGBM Out-Of-Fold (OOF)	0.91388
XGBoost Out-Of-Fold (OOF)	0.91606
Dual Blend Out-Of-Fold	**0.91860**
Public Leaderboard	**0.92587**
Private Leaderboard	0.90298

The Public Leaderboard score of 0.92587 (Figure 7) corroborates the internal validation trajectory, proving that the 5-fold chronological scheme models out-of-time test degradation without overfitting.

8 Reproducibility

The pipeline is contained within standalone script and notebook formats.

8.1 Local Execution

Dependencies are installed via `requirements.txt`:

```
pip install -r requirements.txt
python src/kaggle_ieee_fraud.py
```

8.2 Kaggle GPU Execution

Data files reside in the Kaggle competition input path: `/kaggle/input/competitions/ieee-fraud-detection`. Selecting GPU accelerator Tesla T4 x2 and executing `src/kaggle_ieee_fraud.ipynb` produces `submission.csv`.

9 Conclusion

Developing an effective fraud detection pipeline on non-stationary financial data requires chronological cross-validation. Constructing proxy user identities via `card1` and `addr1` interactions, calculating global aggregations, and retaining native missing values provide critical discriminatory power. Blending multithreaded LightGBM and CUDA GPU XGBoost establishes a validated Out-of-Fold AUC of 0.91860 and a Public Leaderboard AUC of 0.92587.

References

[1] Vesta Corporation and IEEE-CIS, *IEEE-CIS Fraud Detection Competition*, Kaggle, 2019. https://www.kaggle.com/c/ieee-fraud-detection

[2] G. Ke, Q. Meng, T. Finley, et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *NeurIPS*, 2017.

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *ACM SIGKDD*, 2016.